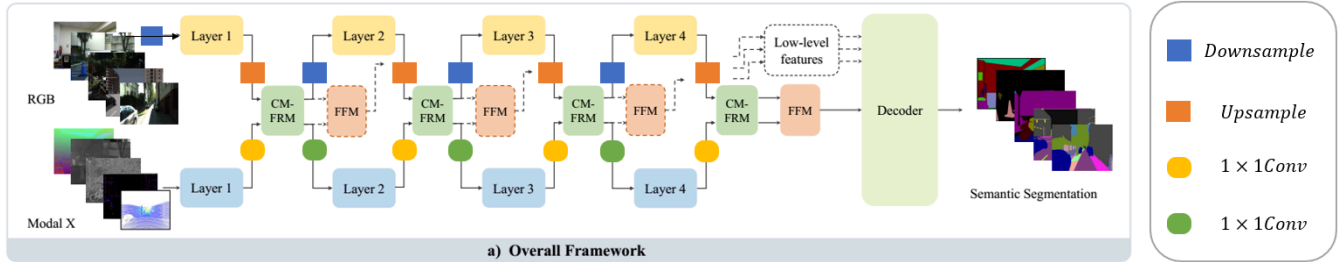


DoubleMiT实验记录

网络结构

BaseLine



Downsample和Upsample均采用双线性插值实现；

1. *Downsample*用于将图像或feature map下采样一定倍率：

$$F_{out} = \text{Downsample}(F_{in}),$$

$$F_{in} \in R^{C \times H \times W}, F_{out} \in R^{C \times \gamma H \times \gamma W}, \gamma \in [0.5, 0.6, 0.7, 0.8, 0.9, 1.0]$$

2. *Upsample*用于将feature map上采样一定倍率：

$$F_{out} = \text{Upsample}(F_{in}),$$

$$F_{in} \in R^{C \times \gamma H \times \gamma W}, F_{out} \in R^{C \times H \times W}, \gamma \in [0.5, 0.6, 0.7, 0.8, 0.9, 1.0]$$

3. 对RGB数据和Depth数据采用非对称backbone可能会导致同一个layer输出特征的通道数不一致，因此采用 $1 \times 1 \text{Conv}$ 用于对齐通道；

细节信息

Feature Fusion

不同的Backbone对应的特征融合模块的参数数量和计算量（不包括用于调整depth map的channel的conv）如下表所示：

Backbone	Channels	Flops(G)	Params(M)
b0_b0	[32, 64, 160, 256]	1.74	4.16
b1_b0	[64, 128, 320, 512]	6.9	16.59
b2_b0	[64, 128, 320, 512]	6.9	16.59

Decoder

使用MLPDecoder, embed_dim=512;

Backbone	Channels	Flops(G)	Params(M)
b0_b0	[32, 64, 160, 256]	21.17	1.33
b1_b0	[64, 128, 320, 512]	21.78	1.6
b2_b0	[64, 128, 320, 512]	21.78	1.6

实验数据

实验细节

数据集为NYUDepthV2，损失函数、优化器等均不变；除此之外，相比于上次实验，这次做出以下改动：

1. 固定随机种子为12345；
2. 单尺度推理；

实验结果

Backbone	分辨率降低比例	mIoU	Params(M)	Flops(G)	FPS
b1_b0	0.5	46.78	34.91	34.05	48.39
b1_b0	0.6	47.98	34.91	35.11	47.88
b1_b0	0.7	47.97	34.91	36.43	48.51

Backbone	分辨率降低比例	mIoU	Params(M)	Flops(G)	FPS
b1_b0	0.8	48.23	34.91	37.87	48.84
b1_b0	0.9	48.23	34.91	39.59	48.45
b1_b0	1.0	48.51	34.91	41.41	48.03
b2_b0	0.5	51.47	45.96	36.4	38.8
b2_b0	0.6	51.89	45.96	38.49	40.13
b2_b0	0.7	52.12	45.96	41.03	39.79
b2_b0	0.8	52.82	45.96	43.88	39.36
b2_b0	0.9	53.29	45.96	47.2	39.35
b2_b0	1.0	53.47	45.96	50.81	38.83

分析与疑惑

关于训练：

- 1. 评估若采用多尺度，单次评估的平均时间约为220s；改为单尺度后，降低为约180s；但仍然很慢，因为训练阶段单个epoch的平均时间不超过30s；
- 2. 对评估阶段的各段代码花费时间进行了测量，发现主要慢在这一步：

```
images = minibatch["data"].cuda()
```

但是在cuda()中将non_blocking设为True并不能提高整体评估效率；

- 3. 为了减少评估所需时间，进行了以下尝试，但均无明显效果：
 - 3.1. 改变评估时的batch_size；
 - 3.2. 使用另外的cuda.Stream()分离出评估时的数据加载任务；
 - 3.3. 将DataLoader的num_workers由16调整为8；
 - 3.4. 删去完成一轮epoch后、转入评估阶段之前的torch.cuda.empty_cache();

关于测速：

- 1. 该文档记录的测速数据均在服务器无其他任务时完成，但对同一backbone的相同分辨率降

低比，测速结果仍有明显波动；

- 2. 上一次的测速脚本中没有model.eval()这一步操作，导致整体结果偏慢；
- 3. 只测出了变backbone时的推理速度变化，未测出变分辨率时的速度变化；
- 4. 尝试另写了一个测速脚本，把送入model的数据从随机初始化的数据变为数据集中的数据，测得的帧率明显降低，显然未排除数据加载时间的干扰；

接下来的实验方向

在ConvNeXtV2上进行几组实验（目前考虑RGB-D backbone做N_A、T_A、B_A三组）；

找一个小一点的Feature-Fusion Module；

找一个小点的Decoder（降低MLPDecoder的embed_dim或换个别的，现在正在找）

Depth分支的Backbone不用预训练，改成随机初始化试试？（重读了一下DFormer，结合之前看的workshop那篇的代码在depth分支中没加载预训练backbone，觉得可能有效？）

找一个其他的测速脚本，看看对测速结果有没有改善；

附录1：FPS多次测量结果

rgb_d_ratio	①	②	③	④	⑤	⑥	⑦	⑧	⑨	
b1_b0_0.5	46.56	40.16	47.6	46.55	39.33	40.19	46.79	46.46	46.54	47.0
b1_b0_0.6	40.09	39.95	39.2	39.28	39.32	39.0	46.45	39.76	46.47	39.0
b1_b0_0.7	47.13	47.39	47.57	46.18	47.6	48.18	39.84	46.6	47.38	47.0
b1_b0_0.8	39.46	40.06	47.73	48.71	47.79	46.69	46.51	47.75	48.33	40.0
b1_b0_0.9	39.56	39.58	39.7	39.79	47.05	47.05	46.65	48.13	40.02	39.0
b1_b0_1.0	47.05	39.84	47.73	46.3	40.37	39.56	39.51	40.19	47.57	40.0
b2_b0_0.5	38.06	31.88	38.76	32.0	31.19	38.37	38.61	32.06	37.69	38.0
b2_b0_0.6	31.62	37.54	38.6	31.3	38.48	39.0	39.79	38.36	38.95	31.0
b2_b0_0.7	38.45	37.55	31.38	31.49	38.13	32.17	31.51	38.23	38.16	31.0

rgb_d_ratio	①	②	③	④	⑤	⑥	⑦	⑧	⑨	
b2_b0_0.8	30.8	31.71	38.42	38.8	38.06	37.59	37.48	38.85	38.8	3
b2_b0_0.9	32.28	31.23	31.62	37.67	37.72	32.4	32.59	31.67	38.59	38
b2_b0_1.0	38.33	31.56	37.98	31.88	31.8	37.38	31.61	38.43	38.64	37